

SEMANA 1 — Base de datos + arranque del backend

Día 1	#14 Diseñar tabla usuarios #15 Diseñar tabla divisas
Día 2	#16 Diseñar tabla tipos_de_cambio #17 Diseñar tabla favoritos #18 Diseñar tabla historial_de_consultas
Día 3	#13 Crear script SQL DDL completo (consolida #14–#18 en un único archivo ejecutable) #21 Crear índices para performance (usuario_id, divisa_id, created_at)
Día 4	#22 Insertar datos semilla — seed con divisas base (ARS, USD, EUR, BTC, ETH) y usuario de prueba #26 Definir política de migraciones (naming, carpeta /migrations, herramienta)
Día 5	#27 Inicializar proyecto Node.js con estructura MVC (/routes, /controllers, /models, /middlewares, /services) #28 Configurar conexión a PostgreSQL con pool (pg / node-postgres, variables de entorno, .env.example)

SEMANA 2 — Autenticación (Backend + Frontend en paralelo)

	Backend	Frontend
Día 6	#29 Registro de usuario (hash bcrypt, validaciones, respuesta JWT)	#56 Configurar React Router — instalar react-router-dom, definir rutas /login, /register, /dashboard
Día 7	#30 Login con JWT (verificación de credenciales, generación de token, refresh básico)	#56 (cont.) Terminar rutas protegidas — redirigir a /login si no hay token
Día 8	#31 Middleware de autenticación (verificar JWT en headers, adjuntar req.user a cada request protegido)	#57 AuthContext global (guardar token en localStorage, exponer user, login(), logout())
Día 9	Soporte y ajustes a los endpoints de auth según lo que pida el frontend	#58 Página de registro conectada al endpoint real de #29 (email, contraseña, nombre)
Día 10	Testing manual del flujo completo (Postman o Thunder Client)	#59 Página de login + prueba end-to-end: registro → login → token → sesión activa en el contexto

Dependencias críticas

→ #27 y #28 deben estar listos antes del Día 6 — sin servidor no hay auth.

→ #29 y #30 deben responder antes de que el frontend conecte #58 y #59.

→ #57 AuthContext debe existir antes de que cualquier ruta protegida funcione.

Criterios de aceptación

- El script DDL corre sin errores en una base PostgreSQL limpia
- Seed carga divisas base sin conflictos
- POST /auth/register devuelve JWT con datos correctos
- POST /auth/login devuelve JWT o error claro si las credenciales son incorrectas
- El middleware rechaza requests sin token válido (401)
- Desde el frontend, un usuario puede registrarse e iniciar sesión
- La sesión persiste al recargar la página
- Las rutas protegidas redirigen a /login si no hay sesión activa

Queda para Sprint 2 (fuera de este sprint)

- Tablas de notificaciones (#19) y acciones bursátiles (#20)
- Vistas SQL y optimización de queries (#23, #24, #25)
- Recuperación de contraseña (#37)
- Todo lo relacionado con cotizaciones, APIs externas y dashboard